

CS 8803: GPU Project 2 Report

Introduction: this report is organized chronologically, showing my results accompanied by discussion.

Phase 1: Implement Baseline Bitonic Sort on First Avail GPU (*H100*)

```
FUNCTIONAL SUCCESS
Array size      : 10000000
CPU Sort Time (ms) : 2690.039062
GPU Sort Time (ms) : 167.496323
GPU Sort Speed   : 59.702805 million elements per second
PERF PASSING
GPU Sort is 16x faster than CPU !!!
H2D Transfer Time (ms): 93.364738
Kernel Time (ms)      : 59.831936
D2H Transfer Time (ms): 14.299648
```

Goal & Observation: Intended to get a baseline implementation of bitonic sort that worked on arbitrary input sizes. My solution was to use padding to the nearest power of two in order to keep the core algorithm unmodified. Functionally, requirements are met however there is a significant performance optimization opportunity.

Phase 2: Implement Coarse-Grained Optimizations on First Available GPU

- Optimized padding implementation such that only the device memory is allocated to a power of two, then an arb sized host array is copied in, then memsets remaining bytes of device buffer to 0x7F. The H2D time was cut by 100x, down to ~8ms. Initially tried to launch a kernel, but the CPU's memset was significantly faster.
- Utilize NSight Compute (ncu) to profile source code. Identified the following bottlenecks
 - kernel is utilizing greater than 80.0% of the available compute or memory performance of the device
 - the difference between calculated theoretical (100.0%) and measured achieved occupancy (76.6%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution.

Phase 3: Description of Individual Optimizations

- Register Swapping: In an attempt to speed up swapping operations, I tried to introduce two registers to hold memory accessed values then used those to assign to the new indices. In theory this reduces memory accesses by one.
- Shared Memory: In order to implement shared memory, I made a separate kernel that utilized shared memory until the blocks were larger than the shared memory size. Afterwards, another kernel was used to complete merges in shared memory
- Reduced Branch Divergence: Under certain circumstances it was observed that branch conditions could be ignored, therefore a custom kernel for the merge step could be used to reduce kernel divergence. This also addresses the *NSight Compute* suggestion to reduce branch divergence.
- Pinned Memory: Allocating special memory regions in the host which do not get paged out should help improve transfer speeds between the gpu and cpu.
- GPU intrinsics: `__shfl_sync` can handle arbitrary permutations of input lanes allowing for a butterfly network within 32 threads of a warp such that *they can swap without shared or global mem*

Phase 4: Results Collection and Analysis of Hybrid Configurations

I developed each of the aforementioned optimizations and constructed sorting programs that utilized some combination of them. Below, is the progression of configurations that I experimented with before reaching my final solution. Refer to *Phase 3* for detailed descriptions of the optimizations used to construct the following hybrid configurations.

Table 1

	Occupancy	MEPS	Throughput	GPU Sort Time (ms)	Kernel Time (ms)	D2H (ms)	H2D (ms)
baseline.cu	71.54	125.424	93.79	79.7	24.5	43.11	11.4
regswap.cu	71.56	136.107	93.77	73.47	21.2	10.44	41.8

Table 1 contains my results for the code from Phase 1 running on the *L40S*. It is the naive bitonic sort without any optimizations besides register swapping.

Table 2

	Occupancy	MEPS	Throughput	GPU Sort Time (ms)	Kernel Time (ms)	H2D (ms)	D2H (ms)
baseline.cu	71.5	290.99	93.78	38.25	21.68	4.49	12.11
regswap.cu	71.54	289.89	93.78	52.34	35.5	3.93	13.67
smem.cu	71.56	289.89	93.8	45.63	31.9	3.94	9.56
smem_nosync.cu	56.11	201.34	74.63	4.54	30.9	4.17	10.35
smem_div.cu	75.42	305.65	79.78	35.47	20.39	4.08	10.99
smem_t1024.cu	85.8	341.55	91.29	32.42	16.9	4.5	10.95
smem_t512.cu	84.95	347.56	91.3	29.2	15.25	3.94	9.98
shfl_smem0.cu	84.23	379.62	88.7	28.7	13.57	4.15	9.80
shfl_smem512.cu	83.89	349.05	89.04	29.79	14.32	4.09	10.05

Table 2 begins with the baseline code from *Phase 2*, specifically the optimized padding technique. This technique is agnostic of kernel optimizations, so it is present in each configuration *Table 2* holds. Next, register swapping was tested, but it had worse performance than the baseline, so this optimization was discarded from future configurations.

Then, shared memory was added to the baseline. The fourth data row shows an attempt to remove the `cudaDeviceSynchronize()` however this resulted in poor performance. Next, I updated my global merge kernel to reduce branch divergence. This increased performance, so all configurations from 5 onwards included it. After fine-tuning the number of threads there were meaningful performance improvements.

Finally, I tested the `__shfl_sync` intrinsic. This was inspired by the blog post written here: <https://winwang.blog/posts/bitonic-sort/>. The main idea behind the optimization is that we can use threads swapping within a warp to convert the basecase of Bitonic Sort from 2 elements to 32 elements. Swapping within a warp is faster than shared memory since there is special hardware

support. The downside is that we cannot scale this to sequences larger than 32 elements. Hence, the inspiration to look for a hybrid configuration in this project. First, I tried the shuffle followed by the reduced branch divergence global merge. Then, I tried a three level hierarchy that used the gpu intrinsics, shared memory, and finally the reduced branch divergence global merge. Interestingly the two level hierarchy had slightly better performance. Hence, my final submission of `kernel.cu` has the following optimizations: padding (phase 2), reduced branch divergence (phase 3) and gpu intrinsics (phase 3).

Observation: I think my shared memory implementation may have been flawed or could be due for more optimization. In my final submission I had to utilize the global memory access kernel for sequences of 32 and larger. I suspect that this is my largest bottleneck because when I used just shared mem and global memory accesses, I could do sequences of up to 1024 in shared mem before switching to global mem. I was hoping that the three level architecture would bridge the gap better but that was not the case with my implementation of shared memory. With a more robust shared mem kernel, I think I would have seen the three tier architecture perform much better and potentially meet the target threshold of 650 MEPS.

Phase 5: Further Possible Optimizations

There were a number of optimizations that I tried to consider however they broke the assignment guidelines. One was using pinned memory, however my naive implementation would have required changes in the restricted region of code. Next, there was packing the int data types into uint16_t data types which would theoretically double throughput. This would also require modifications outside of the allowed region. Just out of curiosity, I took my submission code and naively applied these optimizations.

	Occupancy	MEPS	Throughput	GPU Sort Time (ms)	Kernel Time (ms)	H2D (ms)	D2H (ms)
streamer.cu	55.55	491	44.52	22	18	4.08	0.003

As it turns out the numbers are not pleasing. Somehow the occupancy and throughput were obliterated. Only a real improvement was seen in the D2H transfer time which is somewhat arbitrary to the main focus of this assignment. The MEPS was slightly better, but broke the assignment requirements and focus for GPU kernel tuning.

Given more time, some of these ancillary optimizations could have been fine-tuned in hopes of better performance or to fit within the constraints of the assignment. Some CPU or compiler optimizations could be considered too. Despite not reaching the performance goals I had intended to achieve, I learned the most by using GPU intrinsics to tune my sorting algorithm since I have not seen this approach anywhere else and I got to build my own custom configuration.